

Heuristic Search — Typed Notes

Typed-text version prepared from the uploaded source PDF. Original terminology and structure are retained as closely as possible.

HEURISTIC SEARCH

HEURISTIC

- A heuristic is a problem-solving approach that uses practical, experience-based techniques to find a good-enough solution efficiently, especially when an optimal solution is difficult or time-consuming to determine.
- It relies on rules of thumb, educated guesses, or approximate methods rather than exhaustive computation.

UNDERSTANDING HEURISTIC SEARCH

- Unlike brute-force search, which checks every possible solution, heuristic search uses smart rules to find the most promising path.
- These rules, called heuristics, help estimate the best way to solve a problem.
- By balancing between trying new options (exploration) and using the best-known options (exploitation), heuristic search solves complex problems more efficiently.

UNDERSTANDING HEURISTIC SEARCH

- Exploration focuses on discovering new solutions.
- Exploitation focuses on making the best use of known solutions.
- A good heuristic search balances both to find the best possible result.

SIGNIFICANCE

Heuristic search is important in AI because it helps find solutions faster, especially when there are too many possible options to check one by one. Instead of testing all possibilities, heuristic methods focus on the most promising paths, making the search process quicker and more efficient.

- Handles Large Problems – Useful when the problem has too many possibilities to check each one.
- Saves Time – Helps AI skip unnecessary steps and find solutions faster.
- Makes Complex Problems Solvable – Some problems are too big for exact methods, but heuristics make them manageable.

COMPONENTS OF HEURISTIC SEARCH

Heuristic search algorithms have five key parts:

- State Space – All possible positions or conditions that could lead to a solution.
- Initial State – The starting point of the search where the problem begins.
- Goal Test – A check to see if we have reached the final solution.
- Successor Function – Defines possible next steps from the current state.

- Heuristic Function – A rule that estimates how close a state is to the goal, helping to focus on the best paths.

These components work together to guide the search and find solutions more efficiently.

WHAT IS A HEURISTIC SEARCH?

- A Heuristic is a technique to solve a problem faster than classic methods, or to find an approximate solution when classic methods cannot.
- A Heuristic (or a heuristic function) takes a look at search algorithms. At each branching step, it evaluates the available information and makes a decision on which branch to follow.
- It does so by ranking alternatives. The Heuristic is any device that is often effective but will not guarantee work in every case.
- This is a kind of a shortcut, as we often trade one of optimality, completeness, accuracy, or precision for speed.

Heuristic Search Techniques in Artificial Intelligence

WHY DO WE NEED HEURISTICS?

- To produce a solution in a reasonable amount of time. It doesn't have to be the best — an approximate solution will do since this is fast enough.
- Reduce the polynomial number for most problems that are exponential, and in situations where we can't find known algorithms.
- Heuristic Techniques may be weak methods because they are vulnerable to combinatorial explosion.

DIRECT HEURISTIC SEARCH TECHNIQUES IN AI

- Other names for these are Blind Search, Uninformed Search, and Blind Control Strategy.
- These aren't always possible since they demand much time or memory.
- They search the entire state space for a solution and use an arbitrary ordering of operations.
- Examples of these are Breadth First Search (BFS) and Depth First Search (DFS).

DEPTH FIRST SEARCH (DFS) IN HEURISTIC SEARCH

- DFS explores as deep as possible before backtracking, but it does not use heuristic information.
- However, it can be modified for heuristic search when combined with heuristics (e.g., Depth-Limited Search or Iterative Deepening A* Search).

Example

- Solving mazes: DFS can be guided by heuristics to move toward the goal faster instead of blindly exploring all paths.
- Game AI (e.g., Chess, Tic-Tac-Toe): DFS can be combined with heuristic evaluation functions in the Minimax algorithm to determine the best possible move.

Limitations

- DFS alone does not guarantee finding the best (optimal) solution.
- If not used with heuristics, DFS may explore unnecessary paths, making it inefficient.

BREADTH FIRST SEARCH (BFS) IN HEURISTIC SEARCH

- BFS explores all nodes at the current level before moving deeper. Like DFS, BFS itself is uninformed, but it can be converted into a heuristic search method when combined with heuristics.

Example

- Pathfinding (e.g., Google Maps, GPS Navigation).
- BFS explores all possible paths, but when combined with heuristics (e.g., estimated distance to the destination), it turns into the A^* , which is an efficient heuristic search technique.
- Social Network Friend Suggestions (e.g., Facebook, LinkedIn).
- BFS is used to find the shortest connection path between users. When combined with heuristics, it prioritizes users who have the most mutual connections.

Limitations

- BFS alone is memory-intensive because it stores all nodes at a given depth.
- Without heuristics, BFS explores all possibilities equally, which can be inefficient for large problems.

COMMON WEAK HEURISTIC SEARCH TECHNIQUES

1. Hill Climbing

- Moves step by step toward a better solution based on a given rule.
- Can get stuck at the wrong answer (local optima).
- Example: Climbing a hill without knowing if it's the tallest one.

2. Simulated Annealing

- Sometimes takes a step back to avoid getting stuck in a bad spot.
- Inspired by the heating and cooling process in metals.
- Example: Used to find the shortest route in a travel problem.

3. Gradient Descent

- Moves in the direction that improves results the fastest.
- Commonly used in training AI models.
- Example: Helps machines learn by adjusting weights in neural networks.

COMMON WEAK HEURISTIC SEARCH TECHNIQUES

1. Random Search

- Tries different solutions randomly without a clear plan.
- Slow but useful when no guiding rule exists.
- Example: Guessing a password randomly.

2. Generate and Test

- Creates possible answers and checks if they work.
- Like trial and error.

- Example: Trying different keys to unlock a door.

WHAT IS A CONSTRAINT SATISFACTION PROBLEM (CSP)?

A Constraint Satisfaction Problem (CSP) is a type of problem where we need to find values for a set of variables while following specific rules (constraints).

Key Components

- Variables → The unknowns that need values. Example: In Sudoku, each cell in the grid is a variable.
- Domains → The possible values each variable can take. Example: In Sudoku, numbers 1 to 9 are the domain.
- Constraints → Rules that restrict possible solutions. Example: In Sudoku, no two cells in the same row, column, or 3×3 box can have the same number.

Where CSPs Are Used

- Puzzles and Games (Sudoku, crossword puzzles)
- Scheduling Problems (Timetable scheduling, exam scheduling)
- AI and Robotics (Pathfinding, decision-making)
- Resource Allocation (Assigning workers to shifts, project planning)

COMPONENTS OF CSPS

CSPs have three main components:

1. Variables

- The unknowns that need values.
- Example: In Sudoku, each cell is a variable that must be filled with a number.
- Types: Boolean (True/False), Integer (Numbers), Categorical (Colors, Shapes).

2. Domains

- The possible values for each variable.
- Example: In Sudoku, each cell can take values from 1 to 9.
- Domains can be finite (limited set of values) or infinite (like real numbers).

3. Constraints

- Rules that define valid solutions.
- Unary Constraints → Apply to a single variable (e.g., a Sudoku cell cannot be empty).
- Binary Constraints → Apply to two variables (e.g., two Sudoku cells in the same row cannot have the same number).
- Higher-Order Constraints → Apply to multiple variables (e.g., all numbers in a row, column, and 3×3 grid must be unique in Sudoku).

CSP APPROACH

- The goal test is decomposed into a set of constraints on variables.

- Checks for violation of constraints before new nodes are generated.
- Must backtrack if constraints are violated.
- Forward-checking looks ahead to detect unsolvability, based on the current values of constraint variables.

CSP EXAMPLE: MAP COLORING

Color a map with three colors so that adjacent countries have different colors.

- Variables: A, B, C, D, E, F, G.
- Values: {red, green, blue}.
- Constraints: “no neighboring regions have the same color”.

CONSTRAINT GRAPH

- Visual representation of a CSP.
- Variables are nodes.
- Arcs are constraints.

The map coloring example is represented as a constraint graph with variables A, B, C, D, E, F, and G.

CSP AS INCREMENTAL SEARCH PROBLEM

Initial state

All (or at least some) variables unassigned.

Successor function

- Assign a value to an unassigned variable.
- Must not conflict with previously assigned variables.

Goal test

- All variables have values assigned.
- No conflicts possible — not allowed in the successor function.

Path cost

- e.g. a constant for each step.
- May be problem-specific.

BACKTRACKING SEARCH FOR CSPS

- A variation of depth-first search that is often used for CSPs.
- Values are chosen for one variable at a time.
- If no legal values are left, the algorithm backs up and changes a previous assignment.
- Very easy to implement: initial state, successor function, goal test are standardized.
- Not very efficient; can be improved by trying to select more suitable unassigned variables first.

HEURISTICS FOR CSP

Most-constrained variable (minimum remaining values, “fail-first”)

- Variable with the fewest possible values is selected.
- Tends to minimize the branching factor.

Most-constraining variable

- Variable with the largest number of constraints on other unassigned variables.

Least-constraining value

- For a selected variable, choose the value that leaves more freedom for future choices.

CSP EXAMPLE: MAP COLORING (CONT.)

Most-constrained-variable heuristic. The source repeats the map diagram across pages 25–30, showing variables A, B, C, D, E, F, and G.